

Reviewer Pack — Minimal Geometric Defect of Affine Varieties and Resolution ...

Entry f3fdfdb1381 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 04:43:45 UTC

Minimal Geometric Defect of Affine Varieties and Resolution Proof Length

Entry ID: f3fdfdb1381 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 04:43:45 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry: Affine Varieties **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

{‘1’: ‘For all CNF formulas, the resolution proof length is upper-bounded by a constant multiple of the minimal geometric defect of their associated affine variety.’, ‘2’: ‘The minimal geometric defect is defined as the minimum number of points in general position on the variety that are not in any line through the origin modulo 2-powers.’, ‘3’: ‘A constructive mapping from CNF to an affine variety can be achieved by associating each clause with a point and constructing the intersection of all these points.’}

Rationale (proposer’s reasoning):

{‘1’: ‘The geometric defect measures the non-triviality of the algebraic structure, which might reveal hidden complexities in the resolution proof process.’, ‘2’: ‘Affine varieties provide a rich source of geometric structures that could be exploited to understand the complexity of solving satisfiability problems.’, ‘3’: ‘This conjecture connects algebraic geometry with computational complexity, potentially leading to new insights into the nature of computational hardness.’}

Taxonomy category: ALGEBRAIC_GEOMETRY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 19339ff1d7d4d7fe

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The resolution proof length for a CNF formula is considered supported if it is within a constant multiple (M) of the minimal geometric defect of its associated affine variety, with no seed producing a ratio exceeding $M + 1$. The criterion is falsified if any seed yields a resolution proof length that exceeds M times the geometric defect by more than 10%.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal geometric defect AND resolution proof length IN algebraic geometry OR complexity theory - CNF formulas AND resolution proof length $\leq$ constant * minimal geometric defect OF affine varieties - constructive mapping CNF $\rightarrow$ affine variety AND intersection of points = geometric defect

Top relevant hits considered: - [<http://arxiv.org/abs/1407.1396v9>] On the material geometry of continuously defective corrugated graphene sheets - [<http://arxiv.org/abs/2601.16334v2>] Algebraic Phase Theory I: Radical Phase Geometry and Structural Boundaries - [<http://arxiv.org/abs/1712.07234v2>] The reflection coefficient for minimal model conformal defects from perturbation theory - [<http://arxiv.org/abs/0802.4323v1>] Non-singular affine surfaces with self-maps - [<http://arxiv.org/abs/2112.1325>] Torus fixed point sets of Hessenberg Schubert varieties in regular semisimple Hessenberg varieties - [<http://arxiv.org/abs/1810.04293v2>] Towards geometric Satake correspondence for Kac-Moody algebras - Cherkis bow varieties and affine Lie algebras of type

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from fractions import Fraction

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    for i in range(rows):
        if matrix[i][i] == 0:
            return None # Singular matrix
        for j in range(i + 1, rows):
            factor = -Fraction(matrix[j][i], matrix[i][i])
            for k in range(cols):
                matrix[j][k] += factor * matrix[i][k]
    rank = sum(1 for row in matrix if any(row))
    return rank

def is_independent(points):
    n = len(points[0])
    A = [[points[j][i] for i in range(n)] for j in range(len(points))]
    rank = gaussian_elimination(A)
    return rank == len(points)
```

```

def construct_affine_variety(clauses, num_vars):
    points = []
    for clause in clauses:
        point = [0] * num_vars
        for var in clause:
            if var > 0:
                point[var - 1] += 1
            else:
                point[-var - 1] -= 1
        points.append(point)
    return points

def resolution_length(clauses):
    stack = clauses[:]
    visited = set()
    while stack:
        clause = stack.pop()
        if len(clause) == 0:
            return 1
        literal = random.choice(clause)
        for other_clause in clauses:
            if -literal in other_clause:
                new_clause = [l for l in other_clause if l != -literal]
                if new_clause not in visited:
                    stack.append(new_clause)
                    visited.add(new_clause)

    return 0

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    num_vars = n
    clauses = []
    for _ in range(n):
        clause = [random.randint(-num_vars, num_vars) for _ in range(3)]
        clauses.append(clause)

    variety_points = construct_affine_variety(clauses, num_vars)
    independent_points = [p for p in variety_points if is_independent([p] + variety_points)]
    geometric_defect = len(independent_points)

    proof_length = resolution_length(clauses)

    M = 2 # Example constant multiple
    conjecture_holds = proof_length <= M * geometric_defect

    return {
        "metric_name": "resolution_proof_length",
        "metric_value": proof_length,
        "instances_tested": n,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else f"Proof length {proof_length} exceeds M * ge
    }

if __name__ == "__main__":

```

```

import sys
seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = (sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))**0.5
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(r["counterexample"] for r in results):
    first_failing_seed = next(s for s, r in enumerate(results) if r["counterexample"])
    print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}")
else:
    print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1c1681bc.py", line 96, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1c1681bc.py", line 77, in run_trial
    proof_length = resolution_length(clauses)
                    ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1c1681bc.py", line 59, in resolution_length
    if new_clause not in visited:
    ~~~~~^
TypeError: unhashable type: 'list'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means the pre-registered support condition could not be unambiguously met. | next: Investigate the cause of the crash and attempt to run the test again.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15805
2	preregistration	ollama_remote	glm4:latest	0	0	9764
3	novelty	ollama_remote	glm4:latest	0	0	8338
4	novelty	ollama_remote	glm4:latest	0	0	9755
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20688
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15543
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11739
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10653
9	judge	ollama_remote	glm4:latest	0	0	12799

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 115084 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/f3fdfdb1381.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f3fdfdb1381.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/f3fdfdb1381.tar.gz` (if generated)